SQL Query Generator

By

Varsha P (230833018020112037)

Vaishnavi M (230833018020112036)

Uma Maheswari C I (230833018020112035)

Suhrudha V (230833018020112034)

of

ARUNACHALA ARTS AND SCIENCE (WOMEN) COLLEGE

Submitted to the

In partial fulfilment of the requirement

for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

MANONMANIAM SUNDARANAR UNIVERSITY

TIRUNELVELI – 627012

MARCH -2026

SQL Query Generator

Natural Language to SQL Conversion System

1. Introduction

Working with databases usually requires knowledge of Structured Query Language (SQL). However, many users who need data insights do not have strong SQL knowledge. Writing complex queries can be difficult, time-consuming, and error-prone.

The SQL Query Generator project addresses this challenge by allowing users to interact with databases using simple natural language. Instead of manually writing SQL commands, users can type requests in plain English, and the system automatically converts them into valid SQL queries.

The application uses the IBM Granite 3.3 2B Instruct model to understand user input and generate appropriate SQL statements. It examines the request, identifies relevant database tables and columns, and constructs a secure query based on the user's intent.

The platform also focuses heavily on security. It filters unsafe commands and prevents malicious SQL injections. Along with query generation, the system provides features for exporting generated queries into files such as PDFs and text documents. Users can also share results through messaging platforms.

With a conversational interface built using Gradio, the system makes database interaction easier and more accessible for both technical and non-technical users.

---

2. Problem Statement

Many organizations store large amounts of information in databases. Extracting meaningful information from these databases usually requires writing SQL queries.

However, several challenges exist:

- Non-technical users cannot easily write SQL queries
- Complex queries require experience and practice
- Manual SQL writing increases the chances of syntax errors
- Security risks such as SQL injection must be managed

To overcome these issues, there is a need for an intelligent system that can understand human language and automatically produce correct and secure SQL queries.

---

3. Objective of the Project

The main goal of this project is to design a system that converts user requests written in natural language into executable SQL queries.

Specific objectives include:

- Interpreting user requests written in plain English
- Detecting appropriate database tables and columns
- Generating SQL queries automatically
- Implementing security checks to prevent unsafe commands
- Providing a user-friendly interface for interaction
- Supporting export and sharing features for generated queries

---

4. Application Scenarios

- **Scenario 1: Identifying Recent User Registrations**

  A marketing analyst may want to find users who registered recently.

  For example, the analyst may type a request such as:

  "Display users who registered last month."

  The system analyzes the sentence and determines that the information is related to the users table. It also recognizes the time condition "last month" and creates a date filter accordingly.

  Based on this interpretation, the system generates a SQL query that retrieves user records created within the previous month.

- **Scenario 2: Filtering High-Value Transactions**

  Financial analysts often need to analyze large transactions.

  A request such as:

  "Show transactions above 1000 in the past week"

  is interpreted by the system. It identifies the transactions table and extracts two important conditions:

  - transaction amount greater than 1000
  - transactions occurring within the last seven days

  The system combines these conditions and produces a structured SQL query.

- **Scenario 3: Generating Reports**

  In many situations, generated queries need to be documented or shared.

The platform allows users to export their generated queries into files. These files can be used for reporting, documentation, or presentations.

Supported export options include:

- PDF reports
- Text documents

Each exported file contains information about the user request, the generated SQL statement, and the timestamp of creation.

➢ **Scenario 4: Preventing Unsafe Queries**

Security is a critical part of database applications.

If a user enters a request that contains harmful SQL commands, the system blocks it immediately. Commands such as DROP, DELETE, or ALTER are recognized as potentially dangerous.

The system also detects suspicious patterns commonly used in SQL injection attacks. When such patterns are found, the system prevents query execution and informs the user.

---

5. System Architecture

The project follows a modular architecture where each component performs a specific task.

The main components include:

1. Natural language processing module
2. SQL generation engine
3. Security validation layer
4. User interface module
5. Export and sharing module

These components work together to process user requests and generate secure SQL queries.

---

6. Technologies Used

Several technologies are combined to implement this system.

- **Python** – Used for developing the application logic
- **IBM Granite 3.3 2B Instruct Model** – Performs natural language understanding

- **Gradio** – Provides the web-based user interface
- **SQLite** – Used as the sample relational database
- **ReportLab** – Generates PDF reports
- **Google Colab** – Used for development and testing

Regular expressions are also used for identifying patterns within user input.

---

7. System Requirements

Software Requirements

To run the system, the following software tools are required:

- Python 3.8 or higher
- Gradio library
- Hugging Face Transformers
- PyTorch
- ReportLab library
- A modern web browser

These dependencies can be installed using the pip package manager.

Hardware Requirements

The hardware requirements for the system are minimal because most processing occurs in the cloud environment.

Typical requirements include:

- Processor equivalent to Intel i3 or higher
- At least 4 GB RAM
- Stable internet connection
- Standard display resolution

---

8. Database Design

The system uses a sample SQLite database that includes multiple tables representing typical business data.

The main tables used in the project are:

- Users
- Orders
- Products
- Transactions

Each table contains several columns describing relevant attributes such as user information, purchase details, product data, and financial records.

---

9. Natural Language Processing Process

- The system processes user requests through several stages.
- First, the user input is analyzed to determine the main topic of the request. This helps the system identify which database table should be used.
- Next, keywords within the sentence are mapped to specific database columns. For example, words like "price" or "total" may correspond to the amount column.
- Finally, the system detects conditions such as date ranges or numeric comparisons and converts them into SQL WHERE clauses.

---

10. User Interface

- The application interface is designed using the Gradio framework.
- Users interact with the system through a chat-style interface where they can type requests in natural language. The interface displays both the user message and the generated SQL query.
- Additional interface features include:
- history of previous queries
- button for generating SQL
- option to copy generated queries
- option to clear conversation history

---

11. Export and Sharing Features

- The system allows users to save or share generated SQL queries.
- Generated queries can be exported as:

- PDF reports for professional documentation
- plain text files for editing and storage
- The application also generates shareable links that allow users to send queries through messaging platforms.

---

12. Testing

- The system is tested using several user inputs to ensure that queries are generated correctly.
- Example test input:
- "Show users who joined last month."
- Expected result:
- A SQL query that retrieves records from the users table filtered by signup date.
- Testing also verifies that unsafe inputs are properly blocked.

---

13. Conclusion

- The SQL Query Generator project demonstrates how artificial intelligence can simplify database interaction. By combining natural language processing with SQL generation techniques, the system allows users to retrieve information from databases without manually writing queries.
- Security mechanisms ensure that generated queries are safe and free from malicious commands. The platform’s conversational interface and export capabilities further enhance usability.
- Overall, the project illustrates the potential of AI-driven tools to make database technologies more accessible to a wider range of users.

---

14. Future Improvements

- Several additional features can be added to improve the system in the future.
- Possible enhancements include:
- Support for complex queries involving multiple tables
- Aggregation functions such as SUM, COUNT, and AVG
- Compatibility with larger database systems like MySQL or PostgreSQL

- Advanced query optimization suggestions
- User authentication and role-based access control

---

15. Conclusion

The SQL Query Generator project demonstrates how artificial intelligence can simplify the process of interacting with databases. By using the IBM Granite 3.3 2B Instruct model, the system is able to understand user requests written in natural language and convert them into structured SQL queries automatically.

This approach removes the difficulty of writing complex SQL commands manually and makes database access easier for users who do not have technical expertise. The system intelligently identifies relevant tables, columns, and filtering conditions based on the user's request, producing accurate and meaningful queries.

Security is also an important aspect of the system. The implemented validation mechanisms help detect and block unsafe SQL commands, ensuring that the database remains protected from potential injection attacks and unauthorized modifications